



WinItalPascal – Core

Libreria di utilità per applicazioni VB.NET WinForms

Autore: ItalPascal

Data: 19/09/2026

Informazioni sul progetto

Snodo: Core

WinItalPascal – Libreria VB.NET WinForms

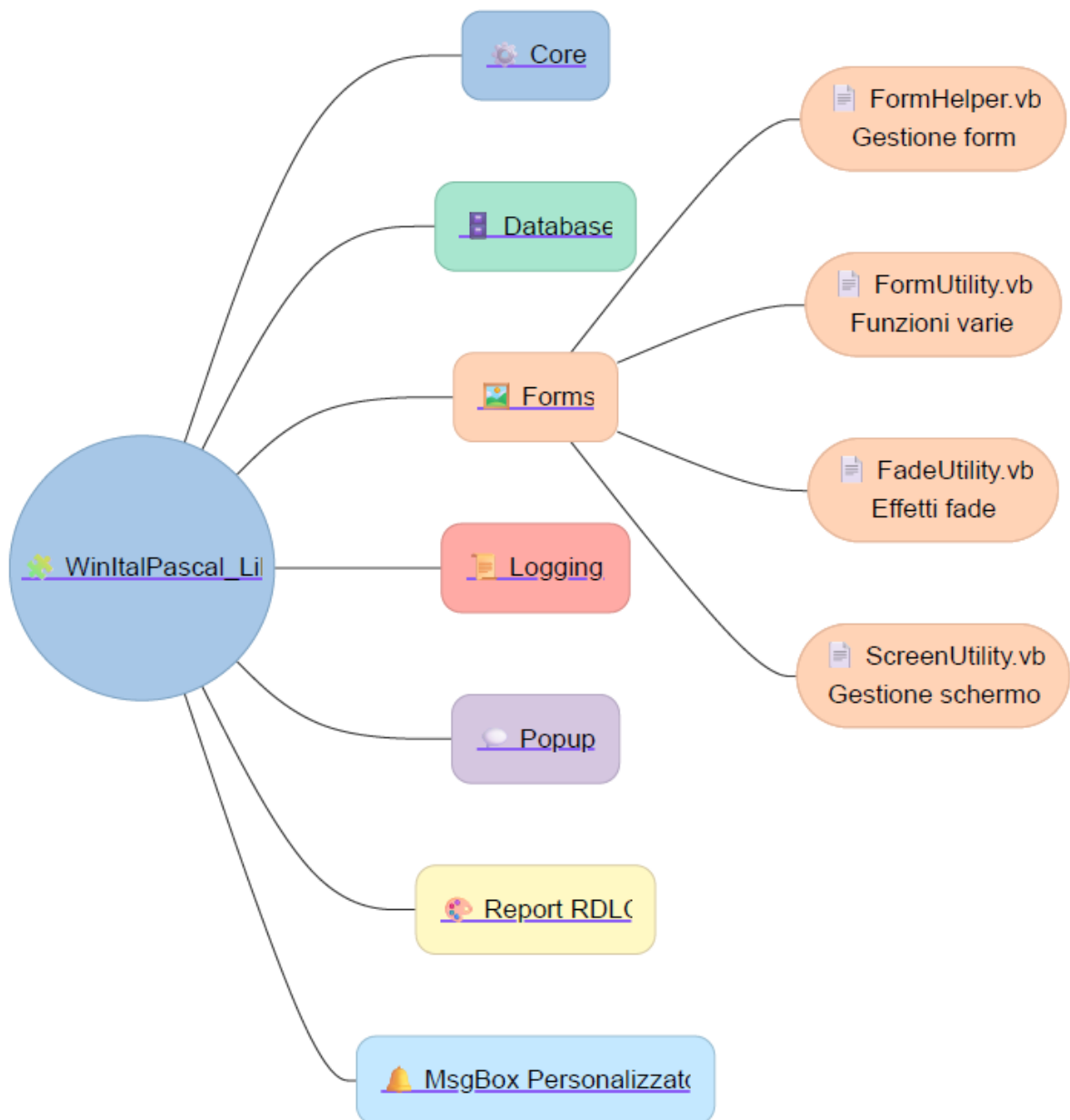
Autore: ItalPascal

Repository GitHub: https://github.com/List051/WinItalPascal_Lib

NuGet: <https://www.nuget.org/packages/WinItalPascal>

Documentazione: <https://list051.github.io/WinVideoShowcase/>

Struttura dello Snodo: Core



Indice

1. FadeUtility.pdf
2. FormHelper.pdf
3. FormUtility.pdf
4. ScreenUtility.pdf

Ital Pascal

WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET

V2.0.6

DOWNLOADS

1.1K

LICENSE

MIT

GITHUB PAGES

ONLINE



FadeUtility – Spiegazione completa

Scopo generale del codice

Il file definisce una **utility statica** (`FadeUtility`) che permette di applicare un semplice effetto di *fade* (apparizione/disapparizione) a qualsiasi controllo di Windows Forms.

L'effetto è ottenuto modificando gradualmente l'**opacità** del colore di sfondo (`BackColor`) del controllo, aumentando o diminuendo il valore alfa in piccoli passi.

Componenti chiave e funzioni

Elemento	Descrizione
<code>Imports System.Drawing</code>	Fornisce la struttura <code>Color</code> e le funzioni di manipolazione dei colori.
<code>Imports System.Windows.Forms</code>	Necessario per accedere a <code>Control</code> , <code>Timer</code> , ecc.
<code>Public NotInheritable Class FadeUtility</code>	Classe "sealed" (non estendibile) che contiene solo metodi statici.
<code>Private Sub New()</code>	Costruttore privato che impedisce l'istanza della classe.
<code>Public Shared Sub FadeControl(ctrl As Control, show As Boolean)</code>	Metodo principale che avvia l'effetto fade sul controllo indicato.
<code>Dim opacity As Double = If(show, 0.1, 1.0)</code>	Valore iniziale di opacità: 0.1 se il controllo deve apparire, 1.0 se deve scomparire.
<code>Dim t As New Timer()</code>	Timer di Windows Forms che genera eventi <code>Tick</code> a intervalli regolari.

Elemento	Descrizione
AddHandler t.Tick, Sub() ... End Sub	Gestore inline (lambda) che viene eseguito ad ogni tick.
If show Then opacity += 0.1 Else opacity -= 0.1	Incremento o decremento dell'opacità.
Math.Max(0, Math.Min(255, CInt(255 * opacity)))	Calcolo dell'intensità alfa (0-255) con limitazione ai valori validi.
ctrl.BackColor = Color.FromArgb(alpha, ctrl.BackColor)	Aggiornamento del colore di sfondo con il nuovo valore alfa.
If opacity >= 1 OrElse opacity <= 0 Then t.Stop()	Arresto del timer quando l'opacità raggiunge i limiti.
t.Interval = 20	Frequenza di tick: 20 ms (≈ 50 fps).
t.Start()	Avvio del timer.

❖ Flusso di esecuzione

Note

1. Chiamata

FadeUtility.FadeControl(Pannel1, True) (o False per far sparire il controllo).

2. Inizializzazione

- Viene impostato opacity a 0.1 (apparire) o 1.0 (scompare).
- Viene creato un nuovo Timer (t).

3. Gestore Tick

- Ogni 20 ms il lambda viene eseguito.
- Se show è True , opacity aumenta di 0.1; altrimenti diminuisce di 0.1.

Ital Pascal

WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET

V2.0.6

DOWNLOADS

1.1K

LICENSE

MIT

GITHUB PAGES

ONLINE

◆ Scopo complessivo

Il file `FormHelper.vb` fornisce un insieme di **utility statiche** per gestire l'apertura, la chiusura e l'aspetto di form all'interno di un'applicazione Windows Forms.

In particolare:

- **Aprire un form all'interno di un container** (ad es. un `Panel`) senza finestre di bordo e con docking completo.
- **Chiudere eventuali form già presenti** nel container prima di aprirne uno nuovo.
- **Creare un titolo personalizzato** per i form, sostituendo la barra del titolo di Windows.
- **Eseguire animazioni di fade-in / fade-out** per rendere l'interfaccia più fluida.

Componenti chiave e funzioni

Funzione	Descrizione	Parametri	Note
<code>ApriForm(frm As Form, container As Control)</code>	Inserisce <code>frm</code> all'interno di <code>container</code> .	<code>frm</code> – form da visualizzare. <code>container</code> – controllo (es. <code>Panel</code>).	Imposta <code>TopLevel = False</code> , rimuove i bordi, esegue <code>Dock = Fill</code> .
<code>CloseOpenForm(container As Control)</code>	Chiude il primo form trovato all'interno di <code>container</code> .	<code>container</code> – controllo che contiene i form.	Attualmente chiude solo il primo form; l' <code>Exit For</code> interrompe l'iterazione.
<code>TitoloPersonalizzato(frm As Form, titolo As String)</code>	Aggiunge un pannello di	<code>frm</code> – form da modificare.	Il pannello è posizionato in

Funzione	Descrizione	Parametri	Note
	intestazione personalizzato al form.	<code>titolo</code> – testo da visualizzare.	alto (<code>Dock = Top</code>) e il form perde la barra di titolo di Windows.
<code>FadeIn(frm As Form, Optional durata As Integer = 20) As Task</code>	Animazione di apparizione (fade-in).	<code>frm</code> – form da mostrare. <code>durata</code> – intervallo in millisecondi tra i passi di opacità.	Incrementa l'opacità di 0,05 per ogni iterazione.
<code>FadeOut(frm As Form, Optional durata As Integer = 20) As Task</code>	Animazione di scomparsa (fade-out).	<code>frm</code> – form da nascondere. <code>durata</code> – intervallo in millisecondi tra i passi di opacità.	Decrementa l'opacità di 0,05 per ogni iterazione.

Flusso di esecuzione

1. Chiamata di `ApriForm`

- Verifica che `frm` e `container` non siano `Nothing`.
- Chiama `CloseOpenForm(container)` per chiudere eventuali form già presenti.
- Configura `frm` (senza bordi, docking completo).
- Aggiunge `frm` al `container`, lo porta in primo piano e lo mostra.

2. Chiamata di `CloseOpenForm`

- Se `container` è `Nothing`, esce.
- Scorre tutti i controlli di `container`.
- Se trova un controllo di tipo `Form`, lo chiude e interrompe il ciclo.

3. Chiamata di `TitoloPersonalizzato`

- Crea un nuovo `Panel` con colore di sfondo blu scuro.
- All'interno, aggiunge un `Label` con il testo fornito, centrato e in grassetto.
- Inserisce il pannello nel form e rimuove la barra di titolo di Windows.

4. Chiamata di `FadeIn` / `FadeOut`

- Imposta l'opacità iniziale (`0` per fade-in, valore corrente per fade-out).
- Mostra o nasconde il form.

- In un loop asincrono, attende `durata` millisecondi, poi incrementa/decrementa l'opacità di `0,05`.
- Termina quando l'opacità raggiunge `1` (fade-in) o `0` (fade-out).

Possibili miglioramenti / errori

Problema	Descrizione	Suggerimento
<code>CloseOpenForm</code> chiude solo il primo form	L' <code>Exit For</code> interrompe l'iterazione dopo il primo match.	Rimuovere l' <code>Exit For</code> per chiudere tutti i form presenti.
Gestione delle risorse	I form chiusi non vengono esplicitamente <code>Dispose</code> .	Dopo <code>Close()</code> chiamare <code>Dispose()</code> o gestire con <code>Using</code> .
Animazioni di fade	Incremento/decremento fisso di <code>0,05</code> può produrre un numero variabile di iterazioni e non garantisce un tempo di durata costante.	Calcolare il numero di passi in base alla durata desiderata e alla opacità finale.
Thread-safe UI	Le funzioni di fade modificano la proprietà <code>Opacity</code> su un thread diverso da quello UI se chiamate da un contesto non UI.	Utilizzare <code>Invoke</code> o <code>BeginInvoke</code> per garantire l'esecuzione sul thread UI.
Tipi di ritorno	<code>FadeIn</code> e <code>FadeOut</code> restituiscono <code>Task</code> ma non sono <code>Async Sub</code> .	Restituiscono <code>Task</code> correttamente; tuttavia, se non si desidera attendere, si può usare <code>Async Sub</code> con attenzione.
Validazione dei parametri	Nessun controllo sul valore di <code>durata</code> (può essere negativo).	Aggiungere un controllo e lanciare un <code>ArgumentOutOfRangeException</code> se necessario.

◆ Conclusioni

`FormHelper` è un helper utile per gestire l'integrazione di form all'interno di un layout a pannelli, per creare interfacce senza bordi e per aggiungere animazioni di transizione. Con alcune piccole modifiche (chiusura di tutti i form, gestione delle risorse, animazioni

più precise) il codice diventa ancora più robusto e pronto per l'uso in applicazioni Windows Forms di produzione.

Ital Pascal

WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET

V2.0.6

DOWNLOADS

1.1K

LICENSE

MIT

GITHUB PAGES

ONLINE

Analisi del Codice VB.NET

1. Scopo Generale

Il metodo `CTitolo` è un helper statico che aggiunge a un form Windows Forms un *header* personalizzato (pannello di titolo) con:

- Titolo centrato
- Pulsante di chiusura (`X`)
- Pulsante di minimizzazione (`-`)
- Possibilità di trascinare il form trascinando l'header

In pratica sostituisce la barra del titolo nativa con una versione più stilizzata e controllata dall'applicazione.

2. Componenti Chiave

Componente	Descrizione	Dettagli di Implementazione
<code>FrmTitolo</code>	Classe non ereditabile (<code>NotInheritable</code>) con costruttore privato.	Serve solo come contenitore statico per il metodo <code>CTitolo</code> .
<code>CTitolo</code>	Metodo <code>Shared</code> che accetta un <code>Form</code> e una stringa di titolo.	Controlla nullità, evita duplicati, crea e aggiunge controlli.
<code>Panel pnlTitolo</code>	Contenitore per l'header.	<code>Dock = Top</code> , altezza 40, colore di sfondo blu scuro.
<code>Label lblTitolo</code>	Visualizza il testo del titolo.	<code>Dock = Fill</code> , centrato, colore bianco, font personalizzato.

Componente	Descrizione	Dettagli di Implementazione
Button <code>btnChiudi</code>	Pulsante “X” per chiudere il form.	<code>Dock = Right</code> , colore verde scuro, evento <code>Click</code> chiude il form.
Button <code>btnMinimizza</code>	Pulsante “—” per minimizzare il form.	<code>Dock = Right</code> , evento <code>Click</code> imposta <code>WindowState = Minimized</code> .
Eventi di trascinamento	<code>MouseDown</code> , <code>MouseMove</code> , <code>MouseUp</code> su <code>pnlTitolo</code> e <code>lblTitolo</code> .	Memorizzano il punto iniziale in <code>Tag</code> , calcolano la nuova posizione del form e lo spostano.
<code>FrameworkLogger</code>	Logger per gestire eccezioni.	Viene chiamato in <code>Catch</code> per registrare errori.

3. Flusso di Esecuzione

1. Verifica preliminare

- Se `frm` è `Nothing` , il metodo esce subito.
- Se il form contiene già un pannello con nome `pnlTitoloPers` , l'operazione viene annullata per evitare duplicati.

2. Creazione del pannello di titolo

- Viene istanziato `pnlTitolo` con le proprietà di stile (colore, dimensione, docking).

3. Creazione del label del titolo

- `lblTitolo` mostra il testo passato, è centrato e occupa tutto lo spazio rimanente.

4. Creazione dei pulsanti

- `btnChiudi` e `btnMinimizza` sono creati con stile flat, colore verde scuro e testo appropriato.
- Vengono aggiunti i gestori di click: chiusura e minimizzazione.

5. Assemblaggio dei controlli

- `lblTitolo` , `btnMinimizza` , `btnChiudi` vengono aggiunti al pannello `pnlTitolo` .
- Il pannello viene poi aggiunto al form e portato in primo piano.

6. Impostazione dello stile del form

- `FormBorderStyle = None` rimuove la barra del titolo nativa.

7. Gestione del trascinamento

- **MouseDown**: se il pulsante sinistro è premuto, memorizza la posizione relativa (`Tag`) e cambia il cursore.
- **MouseMove**: se il pulsante è ancora premuto e `Tag` è presente, calcola la differenza di posizione e sposta il form.
- **MouseUp**: ripristina `Tag` a `Nothing` e il cursore.

- Gli stessi handler sono collegati sia al pannello che alla label perché quest'ultima copre l'intera area di trascinamento.

8. Gestione eccezioni

- Tutto il blocco è avvolto in `Try...Catch`. In caso di errore, viene registrato con `FrameworkLogger.LogError`.

4. Possibili Miglioramenti / Errori

Problema	Suggerimento
Uso di <code>Tag</code>	<code>Tag</code> è un oggetto generico; è più sicuro usare una variabile locale o un campo privato per memorizzare il punto.
Hard-coding dei colori	I colori vengono recuperati da <code>Colori.GetColor</code> . Se la classe non è disponibile, il codice fallirà.
Gestione del focus	Dopo la chiusura o minimizzazione, il focus potrebbe non tornare al form precedente.
Thread-safety	Il metodo è statico ma non è thread-safe; se chiamato da più thread contemporaneamente, potrebbe creare duplicati.
Accessibilità	I pulsanti non hanno descrizioni per screen reader; aggiungere <code>AccessibleName</code> migliorerebbe l'accessibilità.

◆ Conclusioni

Il codice fornisce un modo rapido per aggiungere un header personalizzato a qualsiasi form Windows Forms, sostituendo la barra di titolo predefinita con controlli più controllabili e stilizzati. La logica è ben organizzata in blocchi commentati, ma potrebbe trarre beneficio da una gestione più robusta dello stato del trascinamento e da miglioramenti di accessibilità.

Ital Pascal

WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET

V2.0.6

DOWNLOADS

1.1K

LICENSE

MIT

GITHUB PAGES

ONLINE

Scopo generale del codice

Il file definisce una **utility** per la gestione della posizione e delle dimensioni di finestre (`Form`) in un'applicazione Windows Forms.

Con i metodi statici (`Shared`) è possibile:

- **Adattare** una finestra alle dimensioni dello schermo corrente (con un margine di sicurezza) e centrarla.
- Impostare una finestra in **fullscreen** sul monitor in cui è attualmente visibile.
- Centrare semplicemente una finestra sullo schermo corrente.

Questa classe è pensata per essere usata come helper, quindi non può essere istanziata (`Private Sub New()`).

Componenti chiave e funzioni

Metodo	Significato	Funzionalità principale
<code>AdattaCentra(frm As Form)</code>	Adatta e centra	1. Controlla che <code>frm</code> non sia <code>Nothing</code>. 2. Recupera lo schermo corrente (<code>Screen.FromControl(frm)</code>). 3. Ottiene l'area di lavoro (<code>WorkingArea</code>) escludendo la taskbar. 4. Imposta un margine di sicurezza (20 px). 5. Rimuove eventuali limiti di dimensione massima (<code>MaximumSize = Size.Empty</code>). 6. Ridimensiona la finestra a: $\text{larghezza} = \text{Max}(300, \text{area.Width} - \text{margine})$$\text{altezza} = \text{Max}(200, \text{area.Height} -$

Metodo	Significato	Funzionalità principale
		margin) 7. Centra la finestra calcolando <code>Left</code> e <code>Top</code> .
<code>FullScreen(frm As Form)</code>	Full-screen	1. Controlla che <code>frm</code> non sia <code>Nothing</code> . 2. Recupera lo schermo corrente. 3. Ottiene l'area di lavoro. 4. Imposta <code>StartPosition = Manual</code> per evitare centrare automatico. 5. Imposta <code>Bounds = area</code> (la finestra occupa l'intera area di lavoro).
<code>CentraForm(frm As Form)</code>	Centra	1. Recupera lo schermo corrente. 2. Calcola <code>Left</code> e <code>Top</code> per centrare la finestra usando <code>WorkingArea</code> .
<code>Private Sub New()</code>	Costruttore privato	Previene l'istanza della classe, rendendola un "singleton" di metodi statici.

Nota: tutti i metodi sono `Shared`, quindi si chiamano con `ScreenUtility.AdattaCentra(Me)` ecc.

Flusso di esecuzione tipico

- Chiamata:** in un form, ad esempio `ScreenUtility.AdattaCentra(Me)` o `ScreenUtility.FullScreen(Me)`.
- Verifica:** il metodo controlla se il parametro `frm` è `Nothing`; in tal caso esce subito.
- Screen:** `Screen.FromControl(frm)` restituisce lo schermo in cui il form è attualmente visibile.
- Area di lavoro:** `scr.WorkingArea` fornisce l'area visibile escludendo la taskbar.
- Calcoli:** in base al metodo, si calcolano larghezza/altezza, posizione `Left/Top` o si imposta `Bounds`.
- Esecuzione:** le proprietà del form vengono aggiornate, modificando la sua dimensione e posizione.
- Gestione errori:** eventuali eccezioni vengono catturate e registrate tramite `FrameworkLogger.LogError`.

```

1      ' ESEMPIO:
2      '
3      ' ScreenUtility.CentraForm(Me)

```



*Possibili miglioramenti / errori

Problema	Descrizione	Suggerimento
Logging ;vedi funzione	<code>FrameworkLogger</code> è incluso nel file <code>WinItalPascal.dll</code>	Importare o sostituire con <code>Debug.WriteLine</code> o <code>Console.WriteLine</code> .
Gestione delle eccezioni	Gli errori vengono loggati ma non propagati.	Se necessario, rilanciare l'eccezione (<code>Throw</code>) o restituire un valore di stato.
Margine hard-coded	Il valore <code>20</code> è fisso.	Rendere il margine un parametro o calcolarlo in base alla DPI.
Dimensioni minime	<code>Math.Max(300, ...)</code> e <code>Math.Max(200, ...)</code> sono arbitrari.	Consentire di specificare dimensioni minime tramite parametri.
FullScreen con bordi	Se il form ha <code>FormBorderStyle</code> diverso da <code>None</code> , la taskbar può ancora essere visibile.	Impostare <code>FormBorderStyle = FormBorderStyle.None</code>